



ADVANCED TERRAFORM PRODUCTION HANDBOOK

Architect. Automate. Secure.
Operate at Scale.



Production-Grade
Infrastructure



Team Workflows
& Collaboration



Security, Governance
& Compliance



Multi-Cloud, Multi-Account
& Multi-Region



Reliability, Performance
& Scale



VERIQTA



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



PRODUCTION TERRAFORM ARCHITECTURE

① TERRAFORM IN PRODUCTION ENVIRONMENTS

- Infrastructure as Code at scale
- Consistent, repeatable, and auditable deployments
- Version-controlled and peer-reviewed
- Automated via CI/CD pipelines
- Secure, reliable, and observable

② CONTROL PLANE VS MANAGED INFRASTRUCTURE

CONTROL PLANE	MANAGED INFRASTRUCTURE
• Where Terraform runs • Holds state • Authenticates to cloud • Plans and applies changes • CI/CD, policies, modules	• Resources created by Terraform • VPCs, subnets, compute, databases, storage, etc. • Lives in cloud accounts • The actual workloads

③ REPOSITORY & ENVIRONMENT ARCHITECTURE

- Git-based workflow
- Separate environments (dev, stage, prod)
- Each environment has its own state
- Environments isolated by accounts or subscriptions
- Modules versioned and reusable

④ ROOT MODULES VS REUSABLE MODULES

- Root modules: environment-specific entry points
- Reusable modules: encapsulate components
- Promote reusability, consistency, and standards
- Version modules and publish to registry
- Keep root modules thin and focused

⑤ STATE BOUNDARIES

- One state per environment
- Separate state for independently deployed components
- Isolate sensitive and high-risk resources
- Avoid massive monolithic states

⑥ DEPENDENCY BOUNDARIES

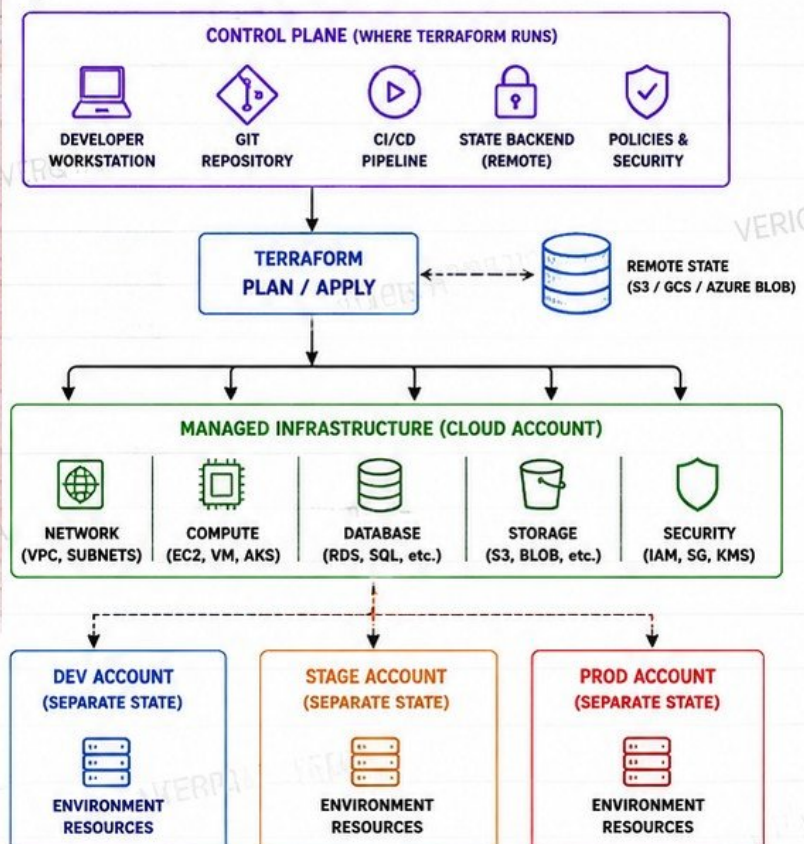
- Define clear dependencies between components
- Use remote state outputs for cross-boundary data
- Minimize inter-state dependencies
- Keep modules loosely coupled

⑦ BLAST-RADIUS REDUCTION

- Small, focused states limit impact
- Least-privilege IAM for deployments
- Separate environments and accounts
- Targeted applies when needed
- Failure in one component does not affect others



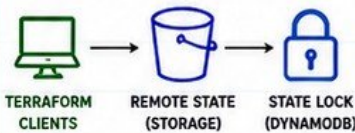
⑧ PRODUCTION ARCHITECTURE DIAGRAM





ADVANCED TERRAFORM STATE MANAGEMENT

① REMOTE STATE ARCHITECTURE



- Store state in remote backend (S3, GCS, Azure Blob, etc.)
- Centralized, secure, and durable
- Enables collaboration and consistency
- Backend handles persistence and locking

② STATE LOCKING



- Prevents concurrent operations
- Uses locking mechanism (e.g., DynamoDB, GCS locking)
- Lock acquired on plan/apply
- Released after operation completes or fails
- Use 'terraform force-unlock' only when safe

③ STATE ISOLATION



- Separate state per environment
- Separate state per team or application
- Never share state across environments
- Reduces blast radius and accidental changes

④ STATE BACKUPS



- Enable versioning for state storage (e.g., S3 versioning)
- Automated daily backups
- Test restore process periodically
- Store backups in separate account or location

⑤ STATE VERSIONING



- Enable versioning on backend storage
- Every change creates a new version
- Roll back to previous state when needed
- Essential for recovery and audit trails

⑥ SENSITIVE INFORMATION IN STATE



- State stores all resource data
- May include sensitive values (passwords, tokens, keys)
- Mark sensitive outputs/variables
- Restrict access to state storage
- Encrypt state at rest and in transit

⑦ STATE CORRUPTION SCENARIOS



- Interrupted writes or network failures
- Manual edits to state file
- Backend or storage failures
- Bug in provider or Terraform version
- Hardware or disk corruption

⑧ RECOVERING PRODUCTION STATE



- Restore from latest backup version
- Verify and validate with 'terraform plan'
- Use 'terraform state rm/import' if needed
- Recreate resources only as last resort

⑨ TERRAFORM STATE COMMAND REFERENCE

COMMAND	PURPOSE	EXAMPLE USAGE
terraform state list	List all resources in the state	terraform state list
terraform state show <address>	Show details of a specific resource	terraform state show aws_instance.web
terraform state mv <src> <dest>	Move resource to a new address in state	terraform state mv aws_instance.old aws_instance.new
terraform state rm <address>	Remove resource from state (not from infrastructure)	terraform state rm aws_s3_bucket.old
terraform import <address> <id>	Import existing infrastructure into state	terraform import aws_s3_bucket.bucket my-bucket-name
terraform state pull	Download current state to a local file	terraform state pull > terraform.tfstate.backup
terraform state push <file>	Push a local state file to remote (use carefully)	terraform state push terraform.tfstate.backup
terraform state replace-provider -old=PROVIDER -new=PROVIDER	Replace provider references in state	terraform state replace-provider -old=hashicorp/aws -new=hashicorp/aws
terraform state force-unlock <lock-id>	Force unlock the state (use with caution)	terraform state force-unlock 1234-5678-90ab-cdef



VERIQTA

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta

STATE SURGERY AND RESOURCE RECOVERY

1 TERRAFORM STATE LIST



- List all resources tracked in the state
- Understand what Terraform knows about your infrastructure

```
terraform state list
# Example:
aws_instance.web
aws_s3_bucket.logs
module.vpc.aws_ycp.main
```

2 TERRAFORM STATE SHOW



- Show attributes for a specific resource
- Useful for inspecting drift or real values

```
terraform state show
aws_instance.web
```

3 TERRAFORM STATE MV



- Move a resource to a new address in state
- Use when refactoring modules or structure

```
terraform state mv
aws_instance.web
module.app.aws_instance.web
```

4 TERRAFORM STATE RM



- Remove a resource from state tracking
- Resource remains in the real infrastructure

```
terraform state rm
aws_s3_bucket.old
```

5 TERRAFORM IMPORT



- Bring existing real infrastructure under Terraform control
- Requires resource ID

```
terraform import
aws_s3_bucket.logs
my-existing-bucket-name
```

6 IMPORT BLOCKS



- Declare import in configuration (Terraform v1.5+)
- Enables repeatable imports

```
import {
  to = aws_s3_bucket.logs
  id = "my-existing-bucket-name"
}
```

7 MOVED BLOCKS



- Declare moved blocks in code (Terraform v1.1+)
- Preserves state during refactors

```
moved {
  from = aws_instance.web
  to   = module.app.aws_instance.web
}
```

8 RECOVERING MANUALLY MODIFIED RESOURCES



- Detect drift with 'terraform plan'
- Decide: adopt change or revert it
- Use import, mv, or state rm as needed
- Document the decision



9 SAFE STATE SURGERY WORKFLOW



1. INSPECT
List and show current state



2. PLAN
Understand the desired change



3. BACKUP
Backup state before changes



4. EXECUTE
Run mv / rm / import carefully

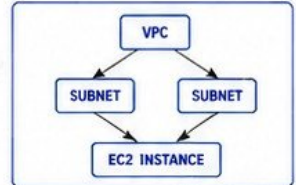


5. VERIFY
Run plan and confirm results

RESOURCE LIFECYCLE AND DEPENDENCY CONTROL

1 TERRAFORM DEPENDENCY GRAPH

- Terraform builds a graph of all resources
- Determines the correct order of operations



2 IMPLICIT DEPENDENCIES

- Created automatically from resource references
- Preferred over explicit dependencies

```
resource "aws_instance" "web" {
  subnet_id = aws_subnet.app.id
  ami       = "ami-123456"
}
# Implicit dependency on aws_subnet.app
```

3 EXPLICIT DEPENDS_ON

- Use depends_on for hidden or external dependencies
- Use sparingly

```
resource "aws_instance" "web" {
  # ...
  depends_on = [aws_iam_role_policy.web]
}
```

4 CREATE_BEFORE_DESTROY

- Create new resource before destroying the old one

```
lifecycle {
  create_before_destroy = true
}
```

5 PREVENT_DESTROY

- Prevent accidental destruction of critical resources

```
lifecycle {
  prevent_destroy = true
}
```

6 IGNORE_CHANGES

- Ignore changes to specific attributes to avoid unnecessary updates

```
lifecycle {
  ignore_changes = [tags, user_data]
}
```

7 REPLACE_TRIGGERED_BY

- Force replacement when another resource or attribute changes

```
lifecycle {
  replace_triggered_by = [
    aws_launch_template.app
  ]
}
```

8 PRECONDITIONS

- Validate conditions before resource creation

```
lifecycle {
  precondition {
    condition = var.instance_type != "t2.micro"
    error_message = "t2.micro is not allowed"
  }
}
```

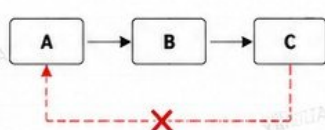
9 POSTCONDITIONS

- Validate conditions after resource creation

```
lifecycle {
  postcondition {
    condition = self.instance_state == "running"
    error_message = "Instance must be running"
  }
}
```

10 AVOIDING DEPENDENCY CYCLES

- Cycles cause plan/apply failures
- Do not create circular references
- Break cycles using data sources, variables, or architectural changes





PRODUCTION MODULE ARCHITECTURE

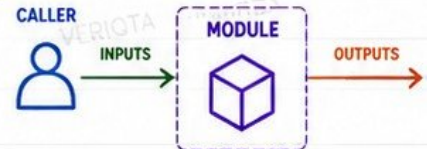
1 DESIGNING REUSABLE MODULES



- Encapsulate infrastructure patterns
- Promote reusability and consistency
- Keep modules focused and single-purpose
- Make modules configurable and flexible
- Document purpose, usage, and examples
- Test modules before publishing

2 MODULE INTERFACES

- Clear contract between module and caller
- Define expected inputs
- Define exposed outputs
- Keep interfaces stable and predictable



3 INPUTS AND OUTPUTS

INPUTS (variables.tf)

```

variable "instance_type" {
  type = string
  default = "t3.micro"
}

variable "tags" {
  type = map(string)
  default = {}
}
  
```

OUTPUTS (outputs.tf)

```

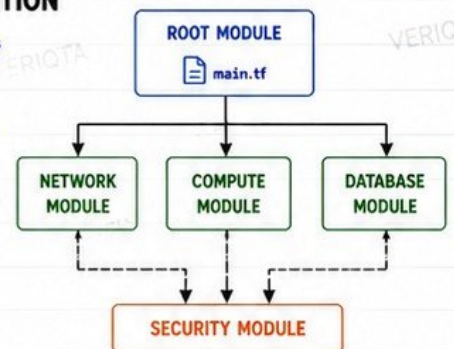
output "instance_id" {
  value = aws_instance.app.id
}

output "instance_ip" {
  value = aws_instance.app.public_ip
}
  
```

★ Outputs expose only what callers need. Avoid leaking internal details.

4 MODULE COMPOSITION

- Build higher-level modules by composing lower-level modules
- Pass outputs as inputs to other modules
- Maintain loose coupling
- Keep modules reusable in different contexts



5 AVOIDING OVERLY COMPLEX MODULES



TOO COMPLEX

- One module = one responsibility
- Avoid "do-everything" modules
- Smaller modules are easier to test, maintain, and version
- Split when module grows in size or scope



SIMPLE & FOCUSED

6 MODULE VERSIONING

- Use semantic versioning (SemVer)
- Tag releases in version control
- Pin module versions in production
- Review changelog before upgrading

SEMVER EXAMPLE

v1.2.3 → Stable release
v1.3.0 → New features (backward compatible)
v2.0.0 → Breaking changes

7 BACKWARD COMPATIBILITY

- Avoid breaking changes in minor/patch releases
- Deprecate inputs/outputs before removing
- Provide migration guide for major versions
- Maintain compatibility for existing consumers

EXAMPLE: DEPRECATION

```

variable "old_name" {
  type = string
  default = null
  description = "(Deprecated)  
Use new_name instead."
}
  
```

8 PROVIDER CONFIGURATION

- Do not hardcode provider credentials
- Accept providers from the root module
- Support provider aliases
- Keep modules cloud-agnostic when possible

USING PROVIDERS

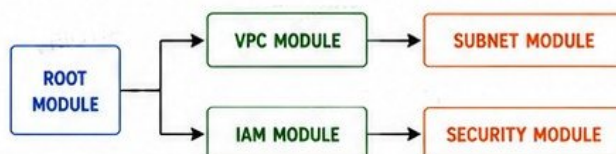
```

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }
}

provider "aws" {
  alias = "usw2"
  region = "us-west-2"
}
  
```

9 MODULE DEPENDENCY MANAGEMENT

- Manage dependencies between modules carefully
- Use explicit dependencies when needed
- Avoid hidden or circular dependencies
- Document module dependencies clearly



BEST PRACTICES

- ✓ Clear dependency flow
- ✓ Minimal coupling
- ✓ Reusable anywhere

ADVANCED HCL AND DYNAMIC INFRASTRUCTURE

① COMPLEX VARIABLE TYPES



- Use rich types for safer, clearer modules
- Prefer strong typing over 'any'
- Validate inputs early

```
variable "config" {
  type = object({
    name = string
    size = number
    tags = map(string)
    ports = list(number)
  })
}
```

② OBJECTS AND MAPS

- Group related values with objects
- Key/value collections with maps



```
variable "settings" {
  type = object({
    env = string
    region = string
    tags = map(string)
  })
}
variable "tags" {
  type = map(string)
  default = {
    team = "platform"
    env = "prod"
  }
}
```

③ FOR EXPRESSIONS

- Transform and build collections
- Filter and reshape data



```
# List of names to uppercase
locals {
  names = ["app", "db", "cache"]
  upper = [for n in local.names :
    upper(n)]
}

# Filter even numbers
locals {
  evens = [for n in range(1, 10) : n
    if n % 2 == 0]
}
```

④ FOR_EACH

- Create one resource per map or set element
- Stable addressing by key



```
resource "aws_s3_bucket" "b" {
  for_each = var.buckets
  bucket   = each.key
  acl      = "private"
  tags     = each.value.tags
}
# var.buckets = {
#   app = { tags = { env = "prod" } }
#   logs = { tags = { env = "prod" } }
# }
```

⑤ COUNT

- Create multiple identical resources
- Use when keys are not required



```
resource "aws_instance" "web" {
  count          = var.instance_count
  ami            = var.ami
  instance_type = var.instance_type
  tags = {
    Name = "web-${count.index + 1}"
  }
}
```

⑥ DYNAMIC BLOCKS

- Generate nested blocks dynamically
- Ideal for repeatable nested structures



```
resource "aws_security_group" "sg" {
  name = "example"
  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port = ingress.value.from
      to_port   = ingress.value.to
      protocol  = ingress.value.proto
      cidr_blocks = ingress.value.cidr
    }
  }
}
```

⑦ CONDITIONAL EXPRESSIONS

- Return one value or another
- Keep code flexible and reusable



```
locals {
  instance_type = var.env == "prod"
    ? "m6i.large"
    : "t3.medium"
}
```

⑧ LOCALS

- Name intermediate values
- Improve readability and reuse



```
locals {
  common_tags = {
    env = var.env
    team = var.team
    owner = var.owner
  }
  name_prefix = "${var.env}-${var.app}"
}
```

⑨ MERGE, FLATTEN, ZIPMAP, TRY, AND CAN

- Powerful functions for safe and flexible logic
- Handle errors and uncertain data



```
locals {
  merged = merge({ a = 1 }, { b = 2 })
  flat   = flatten([[1,2],[3,4]])
  zipped = zipmap(["a","b"], [1,2])
  val    = try(var.optional.value, "default")
  valid  = can(regex("^prod", var.name))
}
```

⑩ DATA TRANSFORMATION PATTERNS

- Combine and reshape data for infrastructure
- Filter, map, group, and build collections
- Keep logic in locals for clarity



RAW DATA
(INPUT)



TRANSFORM
(LOCALS)



MAP / FILTER
(FOR EXPRESSIONS)



STRUCTURE
(OBJECTS / MAPS)



USE IN
RESOURCES

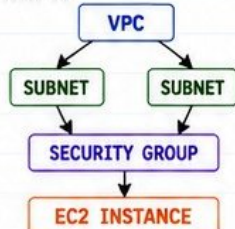
```
# Example: Filter and map instance definitions
locals {
  all_instances = [
    { name = "web", env = "prod" },
    { name = "db", env = "prod" },
    { name = "test", env = "dev" }
  ]
  prod_instances = [
    for inst in local.all_instances :
    inst.name => inst if inst.env == "prod"
  ]
}
```




RESOURCE LIFECYCLE AND DEPENDENCY CONTROL

1 TERRAFORM DEPENDENCY GRAPH

- Terraform builds a graph of all resources and data sources
- Determines the correct order of operations
- Helps identify hidden or unnecessary dependencies



```
# Visualize the dependency graph
terraform graph | dot -Tpng > graph.png
terraform graph | dot -Tsvg > graph.svg
```

2 IMPLICIT DEPENDENCIES

- Created automatically when a resource references another resource
- Based on interpolation and attribute references



```
resource "aws_s3_bucket" "bucket" {
  bucket = "my-bucket"
}
resource "aws_s3_bucket_ownership_controls" "example" {
  bucket = aws_s3_bucket.bucket.id # Implicit dependency
  rule {
    object_ownership = "BucketOwnerPreferred"
  }
}
```

3 EXPLICIT DEPENDS_ON

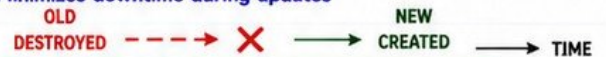
- Use `depends_on` for hidden or external dependencies
- Use sparingly and only when necessary



```
resource "null_resource" "wait_for_db" {
  depends_on = [aws_db_instance.db]
  provisioner "local-exec" {
    command = "echo Database is ready"
  }
}
```

4 CREATE_BEFORE_DESTROY

- Creates the new resource before destroying the old one
- Minimizes downtime during updates



```
resource "aws_lb" "app" {
  name = "app-lb"
  load_balancer_type = "application"
  lifecycle {
    create_before_destroy = true
  }
}
```

5 PREVENT_DESTROY

- Protects critical resources from accidental destruction
- Plan/apply fails if a destroy is attempted



```
resource "aws_db_instance" "db" {
  identifier = "prod-db"
  lifecycle {
    prevent_destroy = true
  }
}
```

6 IGNORE_CHANGES

- Ignore changes to specific attributes
- Useful for externally managed or frequently changing values



```
resource "aws_eks_cluster" "cluster" {
  name = "prod-cluster"
  # ...
  lifecycle {
    ignore_changes = [
      tags,
      desired_capacity
    ]
  }
}
```

7 REPLACE_TRIGGERED_BY

- Forces replacement when another resource or attribute changes
- Useful for rolling deployments and cache invalidation



```
resource "aws_launch_template" "app" {
  name_prefix = "app-lt-"
  image_id = var.ami_id
  lifecycle {
    replace_triggered_by = [
      aws_ami.app_image.id
    ]
  }
}
```

8 PRECONDITIONS

- Validate conditions before resource creation
- Fail fast with clear error messages



```
resource "aws_instance" "app" {
  ami = var.ami_id
  instance_type = var.instance_type
  lifecycle {
    precondition {
      condition = var.instance_type != "t2.micro"
      error_message = "Instance type t2.micro is not allowed."
    }
  }
}
```

9 POSTCONDITIONS

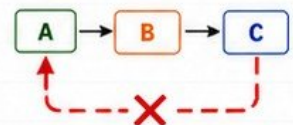
- Validate conditions after resource creation
- Ensures the resource meets expected state



```
resource "aws_instance" "app" {
  ami = var.ami_id
  instance_type = var.instance_type
  lifecycle {
    postcondition {
      condition = self.instance_state == "running"
      error_message = "Instance must be running."
    }
  }
}
```

10 AVOIDING DEPENDENCY CYCLES

- Cycles cause plan/apply failures
- Break cycles by removing unnecessary references
- Use data sources or variables to decouple resources



TIP: Keep modules and resources loosely coupled. Pass IDs/outputs, not whole resources. Review terraform graph to detect cycles early.



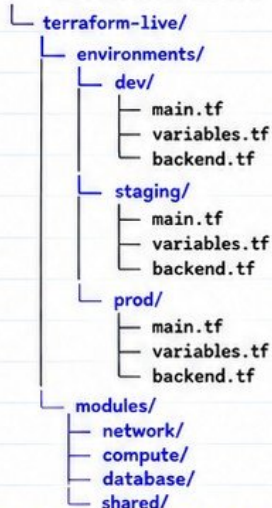
MULTI-ENVIRONMENT ARCHITECTURE

1 DEVELOPMENT, STAGING, AND PRODUCTION

- Separate environments for different stages of the lifecycle
- Promote code through environments
- Isolate changes and failures
- Match production as closely as possible



2 DIRECTORY-BASED ENVIRONMENTS



BENEFITS

- Clear separation
- Independent state
- Environment-specific configuration
- Safer changes
- Easier collaboration

3 SEPARATE STATE FILES

- Each environment has its own state file
- Prevents accidental changes across environments
- Store state in isolated backends
- Use state locking for all environments

ENVIRONMENT	STATE FILE (EXAMPLE)	BACKEND LOCATION
DEV	dev.terraform.tfstate	s3://tf-state/dev/
STAGING	staging.terraform.tfstate	s3://tf-state/staging/
PROD	prod.terraform.tfstate	s3://tf-state/prod/

4 SEPARATE CLOUD ACCOUNTS OR SUBSCRIPTIONS

- Strong isolation between environments
- Limits blast radius
- Separate billing and quotas
- Use different accounts/subscriptions for network, security, and workload isolation



5 VARIABLE MANAGEMENT

- Use separate variable files per environment
- Use .tfvars files (e.g., dev.tfvars, prod.tfvars)
- Use sensitive variables for secrets
- Consider remote variable sources (e.g., SSM, Key Vault)

Example: prod.tfvars

```

region      = "us-east-1"
environment = "prod"
instance_type = "m6i.large"
db_password = "sensitive"
  
```



KEEP SECRETS OUT OF CODE

6 ENVIRONMENT PROMOTION

- Promote the same code through environments
- Test in dev → validate in staging → deploy to prod
- Use version control tags for releases
- Promote with confidence and consistency



7 WORKSPACE LIMITATIONS

- Terraform workspaces are not a full replacement for environments
- Limited isolation and access control
- Easy to mix resources by mistake
- Use workspaces only for truly isolated, simple use cases

GOOD USE CASES

- ✓ Per-tenant deployments
- ✓ Ephemeral environments
- ✓ Simple, isolated resources

NOT RECOMMENDED FOR

- ✗ Prod/Non-prod separation
- ✗ Critical infrastructure
- ✗ Regulatory environments

8 PREVENTING CROSS-ENVIRONMENT CHANGES

- Use separate state backends and credentials
- Restrict IAM permissions per environment
- Protect production with manual approval in CI/CD
- Use policy-as-code to enforce boundaries

9 PRODUCTION ISOLATION PATTERNS

- Dedicated prod account and network
- No direct access from non-prod
- Strict IAM roles and least privilege
- Separate CI/CD pipelines for production
- Audit and monitor all production changes
- Enable guardrails: policies, approvals, and alerts



MULTI-ACCOUNT, MULTI-REGION, AND MULTI-PROVIDER TERRAFORM

1 PROVIDER ALIASES

- Use aliases to define multiple configurations
- Each alias maps to a specific account or region
- Enables safe, explicit targeting of resources

```
provider "aws" {
  alias = "us_east_1"
  region = "us-east-1"
}

provider "aws" {
  alias = "eu_west_1"
  region = "eu-west-1"
}
```

2 MULTIPLE CLOUD ACCOUNTS

- Organize environments across accounts
- Improve security, billing, and isolation
- Separate landing zones per account



3 CROSS-ACCOUNT AUTHENTICATION

- Use IAM roles to access other accounts
- Assume roles instead of long-lived credentials
- Enforce least privilege access

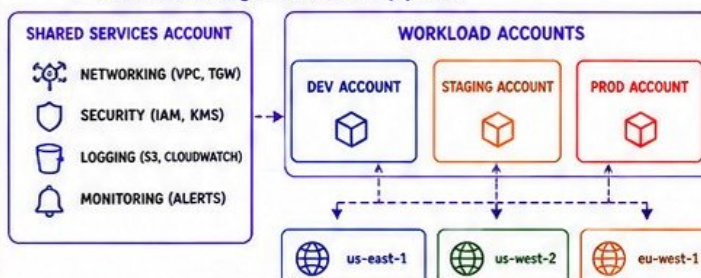
4 MULTI-REGION DEPLOYMENTS

- Deploy resources across multiple regions
- Use provider aliases for each region
- Improve availability and reduce latency



8 DESIGNING GLOBAL INFRASTRUCTURE

- Use multi-account, multi-region patterns
- Centralize shared services
- Standardize networking, security, and logging
- Automate through modules and pipelines



5 PASSING PROVIDERS TO MODULES

- Explicitly pass providers to child modules
- Ensures modules use the correct configuration
- Avoids unexpected provider inheritance

```
module "vpc" {
  source = "../modules/vpc"
  providers = {
    aws.us_east_1 = aws.us_east_1
    aws.eu_west_1 = aws.eu_west_1
  }
}
```

6 PROVIDER INHERITANCE

- Child modules inherit providers automatically
- Override only when a different configuration is needed
- Keep modules flexible and reusable

7 PROVIDER VERSION CONSTRAINTS

- Pin provider versions for consistency
- Avoid unexpected breaking changes
- Use pessimistic constraints (e.g., ~>)

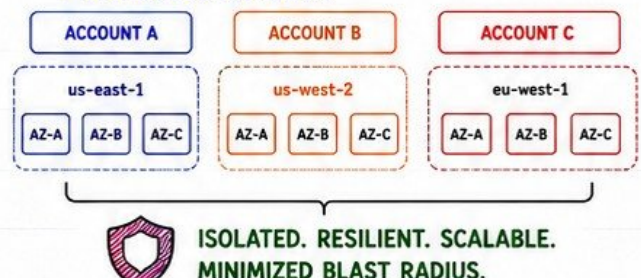
```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```



PREDICTABLE
AND SAFE
DEPLOYMENTS

9 FAILURE-DOMAIN ISOLATION

- Isolate across accounts, regions, and AZs
- Avoid single points of failure
- Design for resilience and disaster recovery
- Limit blast radius of failures

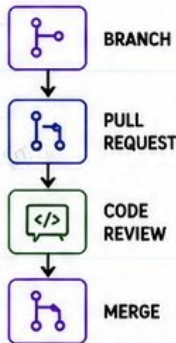




TERRAFORM CI/CD PRODUCTION PIPELINE

1 PULL-REQUEST WORKFLOW

- All changes via pull requests
- Code review before any apply
- Automated checks on every PR
- Merge only after approval
- Keeps infrastructure changes auditable



2 TERRAFORM CODE QUALITY CHECKS

- terraform fmt – consistent formatting
- terraform validate – syntax & config checks
- Fast feedback before planning
- Fail early, fix early

```
terraform fmt -check -recursive # Check formatting
terraform validate               # Validate configuration
```

3 TERRAFORM PLAN

- Generate execution plan for review
- Show changes before any apply
- Detects drift and unintended changes
- Non-destructive and safe

```
terraform init -backend-config=backend.hcl
terraform plan -out=tfplan # Save plan
```

4 PLAN ARTIFACTS

- Store plan files as pipeline artifacts
- Used for approval and apply stages
- Ensures apply matches reviewed plan
- Immutable and traceable



5 APPROVAL GATES

- Manual approval required before apply
- Different approvers for dev/stage/prod
- Policy checks and OPA/Sentinel gates
- Enforces separation of duties



6 PROTECTED PRODUCTION APPLIES

- Production apply is isolated and restricted
- Only approved users or service accounts
- Requires manual approval
- Extra safety for critical infrastructure



7 AUTOMATED VS MANUAL APPLY

AUTOMATED APPLY

- ✓ Dev environments
- ✓ Low-risk changes
- ✓ Feature branches
- ✓ Ephemeral resources

MANUAL APPLY

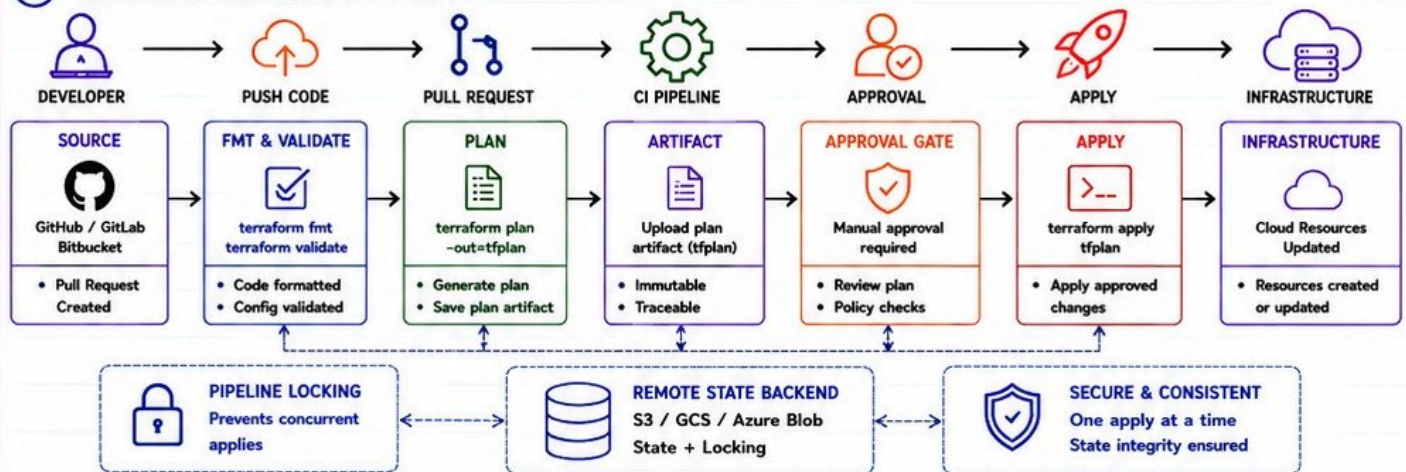
- Staging validation
- Production changes
- High-risk operations
- Critical infrastructure

8 PIPELINE LOCKING

- Prevents concurrent applies
- Uses remote state locking
- Avoids state corruption
- Ensures one apply at a time



9 CI/CD ARCHITECTURE DIAGRAM





SECURE TERRAFORM AUTHENTICATION

1 ELIMINATING LONG-LIVED CREDENTIALS

- Avoid static access keys and passwords
- Use short-lived, automatically rotated credentials
- Reduce risk of key leakage and compromise
- Follow zero-trust authentication principles

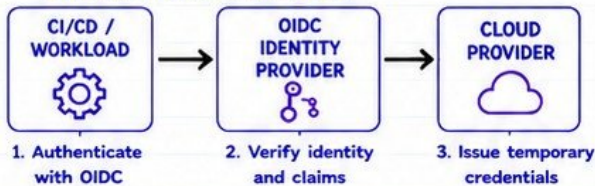
DON'T DO THIS

AWS_ACCESS_KEY_ID = AKIAxxxxxxxxxxxx
 AWS_SECRET_ACCESS_KEY = wJalrXUtnFEMI/K7MDENG
 AWS_SESSION_TOKEN = (empty)



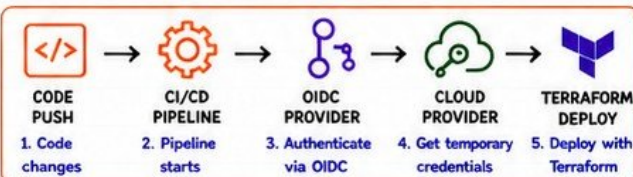
3 OIDC FEDERATION

- Use OpenID Connect (OIDC) for authentication
- CI/CD or workloads authenticate with IDP
- Cloud provider issues temporary credentials
- No static keys stored anywhere



5 CI/CD AUTHENTICATION

- CI/CD authenticates using OIDC or workload identity
- No secrets stored in pipelines
- Cloud provider trusts the OIDC identity
- Pipeline gets temporary credentials to deploy



7 DEVELOPER VS PIPELINE PERMISSIONS

- Developers should not deploy directly to production
- Separate roles for developers and pipelines
- Developers = read/plan only
- Pipelines = deploy (limited scope)

DEVELOPER ROLE	VS	PIPELINE ROLE
- Read resources - Run terraform plan - No apply permissions - No state modification		- Run terraform plan - Run terraform apply - Modify infrastructure - Access state backend

9 AUTHENTICATION FAILURE TROUBLESHOOTING

- Check identity provider configuration
- Verify audience and claims
- Confirm role trust policy
- Check token expiration
- Validate network connectivity
- Review cloud provider logs
- Check permissions and policies
- Use detailed error messages

2 WORKLOAD IDENTITY

- Workload identity replaces static credentials
- Identity is bound to workload, not human users
- Enables fine-grained, auditable access
- Supports cloud-native security best practices

BENEFITS

- ✓ No long-lived secrets
- ✓ Automatic credential rotation
- ✓ Scoped, least-privilege access
- ✓ Stronger security and compliance



4 TEMPORARY CREDENTIALS

- Short-lived credentials reduce attack window
- Automatically expire and are non-reusable
- Used by Terraform to access cloud resources
- Renewed only when needed

EXAMPLE (AWS STS)

```

assume_role_with_web_identity {
  role_arn      = "arn:aws:iam::123456789012:role/terraform-ci"
  role_session_name = "terraform-ci-session"
  web_identity_token = file("/var/run/secrets/oidc/token")
  duration_seconds = 3600
}
  
```



6 LEAST-PRIVILEGE DEPLOYMENT ROLES

- Create roles with only required permissions
- Avoid AdministratorAccess or wide policies
- Scope by account, region, and resource
- Use permission boundaries when needed

EXAMPLE: TERRAFORM DEPLOYMENT ROLE (AWS)

- ✓ Read VPC, Subnets
- ✓ Manage EC2, RDS
- ✓ Manage S3 bucket: tf-state
- ✓ No IAM, No Organizations, No Billing



8 CREDENTIAL ROTATION

- Rotate IAM roles, keys, and secrets regularly
- Use automatic rotation where possible
- Revoke unused credentials immediately
- Monitor and audit credential usage



BEST PRACTICES

- ✓ Use short session durations (<= 1 hour)
- ✓ Use session policies for extra restrictions
- ✓ Audit and alert on unusual access
- ✓ Remove unused roles and keys

ERROR / SYMPTOM	POSSIBLE CAUSE	CHECK / FIX
InvalidIdentityToken	Bad OIDC token or claims	Check token, issuer, audience
AccessDenied	Insufficient permissions	Review IAM role policy
ExpiredToken	Token expired	Reduce session duration or refresh
NoCredentialProviders	No valid auth method	Check environment & providers
Throttling	Too many requests	Retry with backoff



VERIQTA

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



SECRETS AND SENSITIVE DATA



GOLDEN RULE

Never store secrets in Terraform configuration, state, or version control. Use secret managers and least privilege.

Handling secrets correctly in Terraform is critical. Terraform was not designed as a secret management tool.

1 SENSITIVE VARIABLES

- Mark variables as sensitive to hide values in CLI output.
- Does NOT encrypt the value in state.
- Example:

```
variable "db_password" {
  type      = string
  sensitive = true
}
```

Sensitive = hides in UI/CLI output only.

2 SENSITIVE OUTPUTS

- Mark outputs as sensitive.
- Hides values from CLI and UI.
- Still exists in state in plain text.
- Example:

```
output "db_password" {
  value     = aws_db_instance.db.password
  sensitive = true
}
```

3 STATE EXPOSURE RISKS

- State file contains raw resource attributes.
- Anyone with access to state can read secrets.
- Remote state must be encrypted and access restricted.
- Never commit state to Git.

State = Full infrastructure inventory + all secrets in plain text.

4 ENVIRONMENT VARIABLES

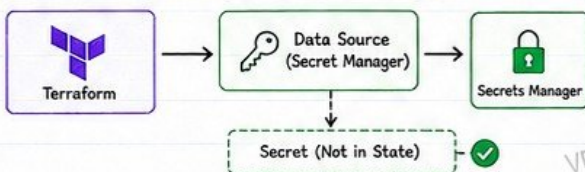
- Pass secrets via environment variables.
- Example:

```
export TF_VAR_db_password="MyS3cret!"
terraform apply
```

Values are not stored in code or tfvars files.

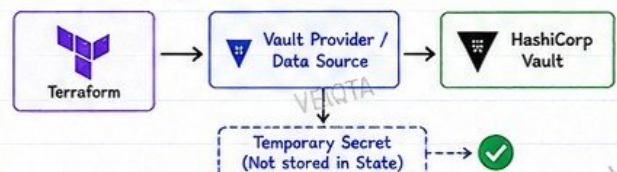
5 SECRET MANAGERS

- Use dedicated tools to store and rotate secrets.
- Examples: AWS Secrets Manager, Azure Key Vault, Google Secret Manager.
- Terraform reads secrets at runtime.
- Secrets stay out of state when used correctly.



6 VAULT INTEGRATION CONCEPTS

- HashiCorp Vault stores and generates secrets.
- Terraform authenticates using short-lived tokens.
- Use the vault provider or data sources.
- Dynamic secrets: generated on demand.
- Never store long-lived credentials.



7 AVOIDING SECRETS IN REPOSITORIES

- Never commit .tfvars with secrets.
- Never commit environment files.
- Use .gitignore to exclude sensitive files.
- Use secret managers and environment variables.
- Scan repositories for secrets regularly.



DO NOT COMMIT

8 CI/CD SECRET HANDLING

- Store secrets in CI/CD secret stores.
- Use masked variables in pipelines.
- Inject secrets as environment variables.
- Limit secret access to only required jobs.



Mask + Protect + Limit Access

9 SECRET LEAKAGE RESPONSE

- Rotate or revoke the leaked secret immediately.
- Identify all systems using the secret.
- Audit access logs and pipeline history.
- Check state files and backups.
- Notify the team and stakeholders if needed.
- Implement prevention to avoid recurrence.





TERRAFORM TESTING AND VALIDATION



TEST EARLY. TEST OFTEN. DEPLOY WITH CONFIDENCE.



1 TERRAFORM VALIDATE

- Checks configuration syntax and internal consistency.
- Validates providers, resources, and arguments.
- Run before every plan or pipeline stage.

terraform
validate



2 TERRAFORM TEST FRAMEWORK

- Built-in testing framework (Terraform 1.6+).
- Write tests in .tftest.hcl files.
- Supports unit and integration testing.

terraform
test



3 .TFTEST.HCL

- Test files use .tftest.hcl extension.
- Define run blocks and assert blocks.
- Human-readable and version-controlled.



4 UNIT-STYLE MODULE TESTS

- Test modules in isolation using mock inputs.
- Validate outputs, resource counts, and values.
- Fast feedback for module developers.



5 PRECONDITIONS & POSTCONDITIONS

- Preconditions run before resource creation.
- Postconditions run after resource creation.
- Prevent bad changes and enforce guarantees.



7 POLICY VALIDATION

- Enforce policies using tools.
- Examples: Sentinel, OPA, Conftest, Checkov, tfsec.
- Ensure compliance and standards.



8 TESTING INFRASTRUCTURE CHANGES

- Always run plan in test environment first.
- Review for destructive changes.
- Validate outcomes after apply.



9 PRODUCTION TEST STRATEGY

- Unit test every module.
- Integration test critical paths.
- Enforce policies in pipeline.
- Use pre/postconditions for safety.
- Test before every production change.

EXAMPLE: .TFTEST.HCL

```
run "example" {
  command = plan
}

assert {
  condition      = length(aws_s3_bucket.this.*.id) > 0
  error_message = "S3 bucket was not created"
}
```

EXAMPLE: UNIT TEST (MODULE)

```
run "module_test" {
  module { source = "../modules/s3-bucket" }
  variables = {
    bucket_name = "test-bucket"
  }
}

assert {
  condition      = aws_s3_bucket.this.bucket == "test-bucket"
  error_message = "Bucket name mismatch"
}
```

INTEGRATION TEST FLOW



PRECONDITION EXAMPLE

```
precondition {
  condition      = var.environment != "prod"
  error_message = "Cannot deploy to prod with this module."
}
```

POSTCONDITION EXAMPLE

```
postcondition {
  condition      = aws_instance.app.state == "running"
  error_message = "Instance is not running"
}
```

POLICY VALIDATION TOOLS



SENTINEL



OPA



CONFTEST



CHECKOV



TFSEC

CHANGE TESTING WORKFLOW



PRODUCTION TEST STRATEGY





POLICY AS CODE AND GOVERNANCE



DEFINE. ENFORCE. PROTECT. GOVERN AT SCALE.

1 INFRASTRUCTURE GOVERNANCE

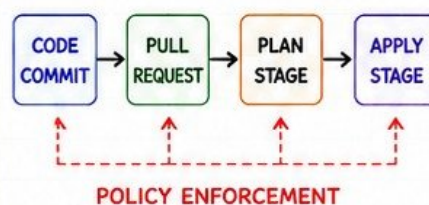
- Ensure consistency, security, and compliance.
- Prevent risky or non-compliant changes.
- Enforce standards across all environments.
- Automate guardrails and reduce human error.



- ✓ COMPLIANCE
- ✓ SECURITY
- ✓ COST CONTROL
- ✓ RELIABILITY

2 POLICY ENFORCEMENT POINTS

- Policies can be enforced at multiple stages.
- Shift security left by failing early.
- Enforce in CI/CD and at deployment time.



3 SENTINEL CONCEPTS

- Sentinel is HashiCorp's policy language.
- Runs in Terraform Cloud / Enterprise.
- Policies-as-code with fine-grained controls.
- Can access plan, config, state, and runtime.
- Supports soft-mandatory and hard-mandatory.

```

import "tfplan/v2" as plan
main = rule {
  all plan.resource_changes as _, rc {
    rc.change.actions contains "delete"
    implies rc.type not in ["aws_s3_bucket"]
  }
}
  
```

4 POLICY TOOLS ECOSYSTEM

OPEN POLICY AGENT (OPA)



- General-purpose policy engine.
- Uses Rego policy language.
- Flexible and extensible.
- Integrates with many tools.

CONFTEST



- CLI tool for policy testing.
- Uses OPA / Rego policies.
- Test JSON/YAML/HCL files.
- Lightweight and fast.

CHECKOV



- Static analysis for IaC.
- Supports Terraform, CloudFormation, Kubernetes, and more.
- Detects misconfigurations.
- Enforces best practices.

TFSEC



- Terraform security scanner.
- Focus on security issues.
- Fast, easy to use.
- Integrates with CI/CD.

5 MANDATORY TAGGING

- Enforce required tags on all resources.
- Ensure accountability and cost tracking.
- Example required tags:

Environment Owner CostCenter Project

```

resource "aws_instance" "example" {
  tags = {
    Environment = "prod"
    Owner       = "platform-team"
    CostCenter  = "12345"
    Project     = "core-app"
  }
}
  
```

POLICY
Deny if any required tag is missing.

6 REGION RESTRICTIONS

- Restrict deployments to approved regions.
- Prevent resources in unsupported regions.
- Example allowed regions:

us-east-1 us-west-2 eu-west-1



ALLOWED

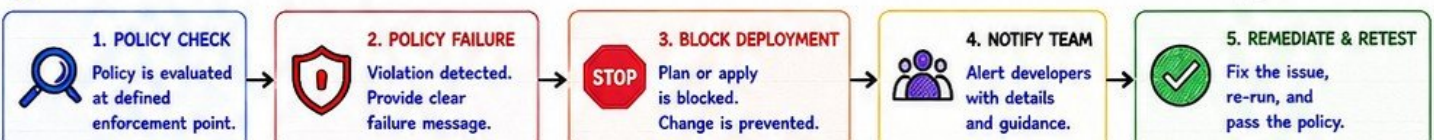
BLOCKED

7 SECURITY GUARDRAILS

- Block unsafe configurations.
- Enforce secure defaults.
- Common guardrails:

- 🔒 Disallow public S3 buckets
- 🔒 Require encryption at rest
- 🔑 Restrict IAM wildcard permissions
- ☁️ Enforce HTTPS only
- 🚫 Disallow root user access

8 POLICY FAILURE WORKFLOW



VERIQTA



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



DRIFT DETECTION AND CONFIGURATION RECONCILIATION



INFRASTRUCTURE CHANGES. DRIFT HAPPENS.
DETECT IT. UNDERSTAND IT. RESOLVE IT.

1 UNDERSTANDING INFRASTRUCTURE DRIFT



- Drift occurs when real infrastructure differs from Terraform configuration.
- Caused by manual changes, external systems, or out-of-band updates.
- Drift can lead to failed applies, unexpected changes, and outages.

2 MANUAL CLOUD CHANGES



- Engineers modify resources outside Terraform.
- Emergency changes are common.
- Third-party services may also change resources.
- These changes create drift.

3 DETECTING DRIFT



- Run `terraform plan` to see drift.
- Terraform refreshes state and compares with real infrastructure.
- Plan shows changes Terraform will make to match configuration.
- Look for unexpected updates or deletions.

4 REFRESH BEHAVIOR



- Terraform refreshes state before planning and applying.
- It reads real infrastructure and updates the state in memory.
- It does NOT write to the state file unless you apply.

5 REFRESH-ONLY PLANS



- Use refresh-only to check drift without proposing changes.
- Command: `terraform plan -refresh-only`
- Shows what has changed in real infrastructure.
- Useful for monitoring and audits.

COMMAND



terraform plan
-refresh-only

6 SCHEDULED DRIFT DETECTION



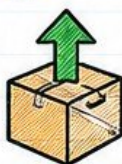
- Run drift detection on a schedule (daily/weekly).
- Use CI/CD pipelines or automation tools.
- Store and review plan outputs.
- Alert on significant or unauthorized drift.

7 DECIDING WHETHER TERRAFORM OR REALITY WINS



- If change is not approved ▶ Terraform wins.
- If change is approved ▶ Update configuration to match reality.
- Consistency is more important than convenience.
- Always follow your team's governance.

8 IMPORTING LEGITIMATE CHANGES



- If manual change is valid, import it.
- Use `terraform import` or import blocks.
- Update configuration to match.
- Commit changes to keep state and code in sync.

COMMAND



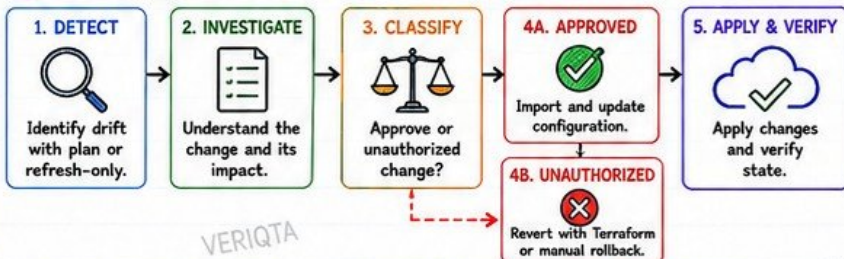
terraform import
<address> <id>

9 REVERTING UNAUTHORIZED CHANGES



- If change is not approved, revert it with Terraform.
- Run `terraform plan` and `terraform apply`.
- Or revert manually and then apply.
- Prevent recurrence with policies and access controls.

10 DRIFT RESPONSE FLOW



KEY TAKEAWAY

Drift is normal. Ignoring it is risky.
Detect early, investigate, decide,
and reconcile with confidence.

BEST PRACTICES

- ✓ Run regular drift detection.
- ✓ Use refresh-only plans for safe checks.
- ✓ Import legitimate changes.
- ✓ Enforce least privilege and policies.
- ✓ Document and communicate all changes.

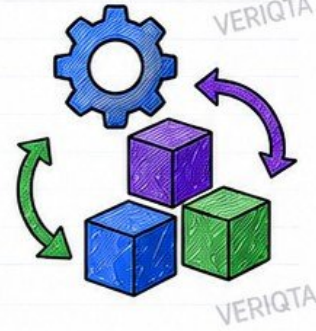
USEFUL COMMANDS

- terraform plan
- terraform plan -refresh-only
- terraform apply
- terraform show
- terraform import <address> <id>
- terraform state list





REFACTORIZING PRODUCTION INFRASTRUCTURE SAFELY



CHANGE STRUCTURE. NOT REALITY.

1 RENAMING RESOURCES

```
resource "aws_s3_bucket"
  "old_name" {
    bucket = "app-bucket"
  }
```



```
resource "aws_s3_bucket"
  "new_name" {
    bucket = "app-bucket"
  }
```

- Renaming changes the address and will destroy/create.
- Use moved blocks to map old address to new address.

```
moved {
  from = aws_s3_bucket.old_name
  to   = aws_s3_bucket.new_name
}
```

2 MOVING RESOURCES BETWEEN MODULES

Before

```
root module
  resource
    aws_db_instance
```



After

```
module.database
  resource
    aws_db_instance
```

```
moved {
  from = aws_db_instance.app
  to   = module.database.
    aws_db_instance.app
}
```

- Moving changes the address (module path).
- Use moved blocks to preserve the resource.

3 CHANGING MODULE STRUCTURE

Before

```
module "network" {
  source = "../modules/network"
  vpc_cidr = "10.0.0.0/16"
}
```



After

```
module "network" {
  source = "../modules/network"
  cidr_block = "10.0.0.0/16"
}
```

- Input/output changes can break existing configurations.
- Use backward-compatible changes and defaults.
- Update in steps and test in lower environments first.

4 MOVED BLOCKS

- Map old addresses to new addresses.
- Prevents resource recreation.
- Works across renames, moves, and module refactors.

```
moved {
  from = module.old_module.aws_subnet.private
  to   = module.network.aws_subnet.private
}
```

Always plan after
adding moved
blocks to confirm
no changes.

5 STATE MIGRATION



Old State



New State

- State migration happens automatically when using moved blocks.
- For complex cases, use terraform state mv before the change.

COMMAND

```
>_
```

```
terraform state mv
<old_address> <new_address>
```

6 IMPORT BLOCKS

- Manage existing resources not yet in state.
- Helps during refactoring or adoption.
- Declare import block and run apply.

```
import {
  to = aws_s3_bucket.logs
  id = "my-existing-bucket"
}
```

Existing Resource
(Not in State)



Managed by Terraform
(Tracked in State)



7 REFACTORIZING WITHOUT RECREATION

Before

```
resource "aws_lb" "app" {
  name = "app-lb"
}
```



After

```
module "lb" {
  source = "../modules/lb"
  name = "app-lb"
}
```

```
moved {
  from = aws_lb.app
  to   = module.lb.
    aws_lb.this
}
```

- Use moved blocks to preserve the resource.
- Verify plan shows NO changes.
- Apply safely with confidence.



Goal: 0 to add, 0 to change,
0 to destroy.

8 REVIEWING DESTRUCTIVE PLANS



DANGER SIGNS

- "-" (Destroy) resources
- Replacement of critical resources
- Unintended cascades
- Data loss risks



ACTIONS

- Understand the change
- Verify moved blocks
- Check dependencies
- Test in lower env
- Get approvals
- Document the change



9 ZERO-DOWNTIME MIGRATION STRATEGY

1. PLAN CAREFULLY



Understand impact
and dependencies.



2. ADD MOVED BLOCKS



Map old addresses
to new addresses.



3. VALIDATE PLAN



Plan must show
0 to destroy.



4. APPLY SAFELY



Apply during low
traffic window.



5. VERIFY & MONITOR



Verify resources
and monitor.



TEST IN NON-PRODUCTION FIRST. COMMUNICATE CHANGES. HAVE A ROLLBACK PLAN.



VERIQTA



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



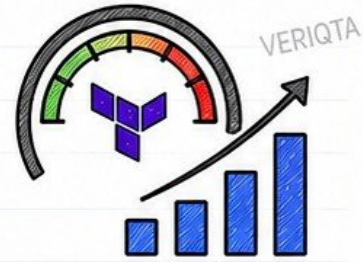
Instagram
veriqta



Telegram
veriqta



TERRAFORM PERFORMANCE AT SCALE



SCALE SMART. RUN FAST. STAY RELIABLE.

1 LARGE STATE PROBLEMS

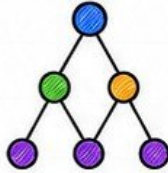


- State file grows very large.
- Slow plan, apply, and refresh.
- High memory and CPU usage.
- Harder to read and debug.
- Higher risk of lock contention.

IMPACT

- Slow operations
- Increased failure risk
- Harder collaboration

2 LARGE DEPENDENCY GRAPHS



- Many resources depend on each other.
- Long planning and apply times.
- Small changes trigger large graph walks.
- Hard to optimize.

IMPACT

- Slow plans and applies
- More chances of failure
- Harder to understand

3 SLOW REFRESH OPERATIONS



- Refresh reads every resource.
- Large environments take a long time.
- Network latency increases refresh time.
- Affects plan accuracy speed.

TIPS

- Use refresh-only plans
- Run during low-traffic windows
- Keep providers up to date

4 PROVIDER API THROTTLING



- Cloud APIs have rate limits.
- Too many requests hit limits.
- Leads to slowdowns and failures.
- Retries increase runtime further.

BEST PRACTICES

- Reduce unnecessary resources
- Use parallelism wisely
- Implement retries and backoff

5 PARALLELISM



- Terraform runs operations in parallel.
- Improves speed but has limits.
- Too high can cause throttling.
- Default is 10 (configurable).

COMMAND

```
>_
terraform apply -parallelism=20
```

6 STATE DECOMPOSITION

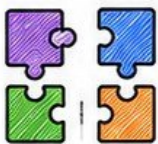


- Split large state into smaller states.
- By environment, domain, or service.
- Improves performance and collaboration.

BENEFITS

- Smaller and faster states
- Less locking and contention
- Easier maintenance

7 MODULE BOUNDARIES



- Design clear module boundaries.
- Keep modules focused and reusable.
- Avoid oversized modules with many resources.

BEST PRACTICES

- One responsibility per module
- Reuse modules across environments
- Keep inputs and outputs clean

8 TARGETED OPERATIONS



- Use -target for emergency or focused changes.
- Speeds up operations.
- Not for regular workflow.
- Risk of incomplete plans.

COMMAND

```
>_
terraform apply -target=module.app
```

9 CI PERFORMANCE



- Optimize CI for speed.
- Use caching and remote state.
- Run in parallel stages where possible.

BEST PRACTICES

- Cache providers and modules
- Use remote backend
- Parallelize test and plan jobs

10 SCALING TERRAFORM ACROSS HUNDREDS OR THOUSANDS OF RESOURCES



1. PLAN FOR SCALE
Design for scale from day one.



2. SPLIT INTELLIGENTLY
Use multiple states and workspaces.



3. OPTIMIZE MODULES
Keep modules small, focused, and reusable.



4. USE REMOTE STATE
Use remote backends for reliability and speed.



5. TUNE PARALLELISM
Find the right balance for your environment.



6. MONITOR & IMPROVE
Measure performance and improve continuously.

GENERAL BEST PRACTICES

- Keep Terraform and providers updated
- Use meaningful outputs and data sources
- Avoid unnecessary data sources
- Clean up unused resources and modules
- Review and optimize regularly

KEY TAKEAWAY

Terraform scales when you design for it. Break down large states, optimize modules, and use smart workflows to keep operations fast, reliable, and predictable.





PRODUCTION FAILURE AND TROUBLESHOOTING PLAYBOOK



**TROUBLESHOOT SYSTEMATICALLY. VERIFY EACH LAYER.
DON'T GUESS. OBSERVE, ISOLATE, AND RESOLVE.**

VERIQTA

VERIQTA

COMMON PRODUCTION FAILURE SCENARIOS

1 STATE LOCK WILL NOT RELEASE



- Stale lock in remote backend.
- Previous run was interrupted.
- Use force-unlock carefully.

2 PROVIDER AUTHENTICATION FAILS



- Expired credentials or tokens.
- Incorrect role or permissions.
- Check environment variables and role.

3 RESOURCE ALREADY EXISTS



- Resource created outside Terraform.
- Import the resource or adjust config.

4 DEPENDENCY CYCLE DETECTED



- Cyclic references in resources or modules.
- Break the cycle using data sources or separate modules.

5 API RATE LIMITS



- Too many API requests.
- Add retries, backoff, or reduce parallelism.
- Split large applies.

6 APPLY PARTIALLY FAILS



- Some resources created, others failed.
- Fix the issue and re-run apply.
- State may already have changes.

7 RESOURCE UNEXPECTEDLY REQUIRES REPLACEMENT



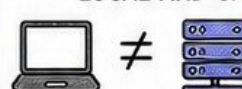
- Change in immutable attribute.
- Review plan carefully before applying.
- Use lifecycle rules where appropriate.

8 PROVIDER CRASHES



- Bugs or memory issues.
- Upgrade provider version.
- Re-run with debug logs for details.

9 PLAN DIFFERS BETWEEN LOCAL AND CI



- Different environment or variable values.
- Different versions of Terraform/provider.
- Check remote state and workspaces.

10 TERRAFORM HANGS DURING REFRESH



- Large state or slow API responses.
- Network issues or provider bug.
- Enable debug logs and investigate.

TROUBLESHOOTING FLOW – CHECK EACH LAYER IN ORDER

1. CONFIGURATION



- ✓ Syntax valid?
- ✓ Variables correct?
- ✓ Provider versions compatible?
- ✓ Backend configured correctly?

2. PROVIDER



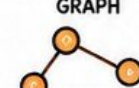
- ✓ Auth working?
- ✓ Correct region?
- ✓ Provider version compatible?
- ✓ Required APIs enabled?

3. STATE



- ✓ State accessible?
- ✓ Lock acquired?
- ✓ State not corrupted?
- ✓ Latest backup available?

4. DEPENDENCY GRAPH



- ✓ Any cycles?
- ✓ Resources ordered correctly?
- ✓ Implicit dependencies working?

5. CLOUD API



- ✓ API reachable?
- ✓ Rate limits?
- ✓ Permissions sufficient?
- ✓ Service health OK?

6. RESOURCE



- ✓ Resource exists?
- ✓ Correct state?
- ✓ Dependencies satisfied?
- ✓ No drift causing issues?



**GO LAYER BY LAYER. DO NOT SKIP STEPS.
FIX THE ROOT CAUSE, NOT JUST THE SYMPTOM.**



USEFUL
COMMANDS

```
terraform validate
terraform plan -refresh-only
terraform state list
terraform state pull
```

```
TF_LOG=DEBUG terraform plan
terraform providers
terraform graph
terraform force-unlock <ID>
```



VERIQTA



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta

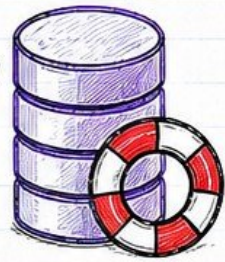


Instagram
veriqta



Telegram
veriqta

FAILED APPLY, ROLLBACK, AND DISASTER RECOVERY



**PREPARE FOR FAILURE. RECOVER FAST.
DOCUMENT EVERYTHING. TEST YOUR RECOVERY.**

1

UNDERSTANDING PARTIAL APPLIES



- Terraform applies resources in dependency order.
- If apply fails, some resources may be created and others not.
- Infrastructure and state can become out of sync.
- Never assume everything is unchanged.

2

WHY TERRAFORM HAS NO TRADITIONAL ROLLBACK



- Terraform makes incremental real-world changes.
- No concept of transactions across providers.
- Rollback could cause more damage.
- Re-run apply to reach desired state.

3

RE-RUNNING FAILED APPLIES SAFELY



- Fix the root cause first.
- Re-run `terraform plan` to confirm.
- Ensure the plan only makes intended changes.
- Re-run `terraform apply`.
- Use `-parallelism` carefully if needed.

4

STATE AFTER PARTIAL FAILURE



- State may contain some new resources.
- Some resources may be missing or tainted.
- Refresh state to see actual infrastructure.
- Use `terraform state list` and `show`.

5

RECOVERING DELETED INFRASTRUCTURE



- If resource deleted outside Terraform, import it back.
- If deleted by mistake, re-create from config.
- Check logs, dashboards, backups, and events.
- Validate dependencies before re-apply.

6

RESTORING STATE BACKUPS



- Restore the most recent good state file.
- Store backups in versioned, encrypted storage.
- Update backend with restored state.
- Run plan to understand drift.
- Apply carefully after validation.

7

PROVIDER / API OUTAGES



- Check provider status page.
- Retry with backoff.
- Reduce parallelism.
- Use cached credentials.
- Wait for service restoration.

8

EMERGENCY MANUAL CHANGES



- Make minimal changes to restore service.
- Document every manual action.
- Prefer non-destructive fixes.
- Communicate with the team immediately.

9

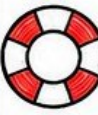
RECONCILING AFTER INCIDENTS



- Refresh and review state vs real infrastructure.
- Import or re-create resources as needed.
- Remove drift and unintended changes.
- Run plan until no unexpected changes.
- Document the incident and resolution.

10

TERRAFORM DISASTER RECOVERY RUNBOOK



- Know where backups are stored.
- Know how to restore state.
- Test recovery regularly.
- Keep runbook updated.
- Practice with game days.

TERRAFORM DISASTER RECOVERY RUNBOOK



DISASTER RECOVERY CHECKLIST

- | | | |
|-------------------------------------|----------------------------------|--------------------------------------|
| ✓ State backups are available | ✓ IAM permissions verified | ✓ Runbook is up to date |
| ✓ Backup restoration tested | ✓ Provider credentials available | ✓ Contacts and escalation documented |
| ✓ Access to cloud accounts verified | ✓ Critical modules tested | ✓ Post-incident review scheduled |

REMEMBER

You can't prevent all failures, but you can prepare, respond, and recover like a pro.



VERIQTA



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

PRODUCTION TERRAFORM TEAM OPERATING MODEL

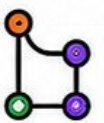


PEOPLE + PROCESS + POLICY + TOOLING = RELIABLE INFRASTRUCTURE
CONSISTENCY. COLLABORATION. CONTROL. CONFIDENCE.

VERIQTA

VERIQTA

1 GIT WORKFLOW



- Trunk-based development.
- Short-lived feature branches.
- Changes via pull requests.
- Merge only after approval.
- Keep main branch always deployment-ready.

2 BRANCH PROTECTION



- Protect main and production branches.
- Require PR reviews and status checks.
- Require up-to-date branches.
- Restrict who can push.
- Prevent force pushes.

3 PULL-REQUEST REVIEWS



- Every change requires review.
- Check plan for unintended changes.
- Verify module versions and providers.
- Ensure policies and standards are met.

4 CODEOWNERS



- Define owners for modules, environments, and files.
- Automatically request reviews from owners.
- Ensure domain experts review changes.

5 INFRASTRUCTURE OWNERSHIP



- Clear ownership per resource or service.
- Owners responsible for lifecycle and reliability.
- Maintain documentation and runbooks.

6 MODULE OWNERSHIP



- Assign owners for each module.
- Module changes require owner approval.
- Maintain module documentation and examples.
- Publish and version modules consistently.

7 VERSION PINNING



- Pin Terraform and provider versions.
- Pin module versions.
- Update versions intentionally and test.
- Avoid uncontrolled upgrades.

8 CHANGE APPROVALS



- Require approvals for all changes.
- Separate plan and apply permissions.
- Production applies require explicit approval.

9 PRODUCTION ACCESS CONTROLS



- Least-privilege IAM roles.
- Separate roles for CI/CD and humans.
- Restrict who can apply to production.
- Use MFA for human access.

10 AUDIT TRAILS



- Log all Terraform operations.
- Keep CI/CD logs and approvals.
- Track who changed what, when, and why.
- Store logs securely and long-term.

11 EMERGENCY CHANGES



- Follow documented break-glass process.
- Make minimal, safe changes.
- Document everything.
- Reconcile with Terraform immediately.

12 PREVENTING CONCURRENT INFRASTRUCTURE MODIFICATIONS



- Use remote state locking.
- Enforce one plan/apply at a time.
- Lock state during operations.
- Avoid manual changes outside Terraform.

TEAM OPERATING WORKFLOW



KEY PRINCIPLES

- ✓ Automate everything possible.
- ✓ Enforce policy and standards.
- ✓ Small changes, reviewed always.
- ✓ Everyone owns quality and reliability.
- ✓ Secure by default, always.
- ✓ Document, communicate, collaborate.

REMEMBER

A strong process prevents mistakes. A strong team recovers from them.



VERIQTA



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



PRODUCTION READINESS CHECKLIST AND COMMAND REFERENCE



CHECK EVERYTHING. AUTOMATE WHAT YOU CAN.
DOCUMENT WHAT YOU DO. PROTECT WHAT YOU DEPLOY.

1 CONFIGURATION



- ✓ terraform fmt -check
- ✓ terraform validate
- ✓ Review provider constraints
- ✓ Review module versions

2 PLANNING



- ✓ terraform plan
- ✓ Review replacements
- ✓ Review deletions
- ✓ Check unexpected drift
- ✓ Save production plans where appropriate

3 STATE



- ✓ Verify remote backend
- ✓ Verify locking
- ✓ Verify backups
- ✓ Restrict state access

4 SECURITY



- ✓ Use temporary credentials
- ✓ Protect secrets
- ✓ Enforce least privilege
- ✓ Run security and policy checks

5 DEPLOYMENT



- ✓ Require peer review
- ✓ Use protected CI/CD
- ✓ Maintain audit records
- ✓ Validate infrastructure after apply

ESSENTIAL COMMAND REFERENCE

1	terraform init	Initialize working directory and download providers.	6	terraform show	Show the current state or a saved plan.	11	terraform state rm	Remove a resource from the state.
2	terraform fmt	Reformat configuration files.	7	terraform output	Show output values from the state.	12	terraform import	Import existing infrastructure.
3	terraform validate	Validate configuration syntax and structure.	8	terraform state list	List all resources in the state.	13	terraform refresh	Update state with real infrastructure.
4	terraform plan	Create and show an execution plan.	9	terraform state show	Show details of a specific resource.	14	terraform force-unlock	Manually unlock the state if locked.
5	terraform apply	Apply the changes required to reach desired state.	10	terraform state mv	Move a resource in state.	15	terraform graph	Generate a dependency graph.

16 terraform test

Run Terraform tests (.tftest.hcl).



PRODUCTION READINESS REMINDERS

- ✓ Automate and enforce standards.
- ✓ Test changes before production.
- ✓ Keep state safe and backed up.
- ✓ Use least privilege always.
- ✓ Review every change.
- ✓ Document everything.
- ✓ Monitor and audit continuously.
- ✓ Plan for failure and recovery.

REMEMBER

Infrastructure is code.
Treat it with the same care as application code.
Build it right. Run it safe.
Keep it reliable.



VERIQTA



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta